

Tide retrospective: *shifted-affine κ_3 for the anharmonic Gibbs (G2 follow-up)*

Daniel Murfet

7 May 2026

Abstract

This retrospective records a 384-line shifted-affine extension of the G2 tide.¹ G2 proved $t^2 \cdot \kappa_3(bx, ax, cx) \rightarrow -(abc)\alpha/\lambda^3$ for the 1D anharmonic Gibbs measure with potential $L = (\lambda/2)x^2 + (\alpha/6)x^3 + (\gamma/24)x^4$. Its retrospective flagged the shifted-affine extension $\kappa_3(a_1x + b_1, a_2x + b_2, a_3x + b_3) = a_1a_2a_3 \cdot \kappa_3(x, x, x)$ as a deferred follow-up, estimated ~ 50 extra lines for the constant-observable plumbing. This tide lands that follow-up. The actual cost was higher — ~ 270 lines of new content, driven by integrability bookkeeping needed when constant terms in the integrand make I2’s additivity lemma load-bearing (rather than just the scalar-pull lemma, which sufficed for G2’s unshifted case). Single session; build green on the first attempt after a small rewrite re-targeting; zero sorry, axiom, or native_decide. The numerical sanity check GPT-5.5 Pro recommended (the *exact* finite- t identity, not just the asymptotic) confirmed the multilinear factorisation to machine precision over a five-by-four parameter sweep.

1 Setting

The base G2 tide proved that for any scalars $a, b, c \in \mathbb{R}$,

$$\kappa_3(bx, ax, cx) = (abc) \cdot \kappa_3(x, x, x)$$

as a static identity in any potential L , then specialised to the anharmonic Gibbs measure to recover Tide 9’s asymptotic with an extra abc factor. The proof routed every `gibbsExp` call through Tide 9’s $h = 0$ bridge to `Laplace.gibbsExpectation`, then peeled scalars one by one via I2’s `gibbsExpectation_smul`.²

The retrospective for that tide named the shifted extension as a follow-up that “defer until a second consumer for shifted observables materialises” but estimated only ~ 50 lines. The estimate was optimistic by a factor of five, for a structural reason worth recording.

2 The thing formalised

For the 1D anharmonic potential $L_{\lambda\alpha\gamma}(x) = (\lambda/2)x^2 + (\alpha/6)x^3 + (\gamma/24)x^4$ with $\lambda > 0$, $\gamma > 0$, $\alpha^2 < 3\lambda\gamma$, and arbitrary scalars $a_1, a_2, a_3, b_1, b_2, b_3 \in \mathbb{R}$:

$$t^2 \cdot \kappa_3(\text{vol}, L, a_2x + b_2, t, a_1x + b_1, a_3x + b_3) \longrightarrow a_1a_2a_3 \cdot (-\alpha/\lambda^3) \quad \text{as } t \rightarrow \infty.$$

¹G2 base linear case: branch `tide/kappa3-affine-anharmonic`, commit `1b3e9bd`.

²`gibbsExpectation_smul L t c  $\varphi$  :  $\langle c\varphi \rangle = c\langle \varphi \rangle$` is unconditional, no integrability needed — that is why the unshifted case lands cleanly in ~ 100 lines.

The constants b_i drop because the third joint cumulant vanishes on constant slots; the multipliers a_i pull through by trilinearity.

The Lean signature:

```
theorem kappa3_anharmonic_shifted_affine_asymptotic
  {lam alpha gamma : ℝ}
  (hlam : 0 < lam) (hgamma : 0 < gamma)
  (hdisc : alpha ^ 2 < 3 * lam * gamma)
  (a a a b b b : ℝ) :
  Filter.Tendsto
    (fun t : ℝ => t ^ 2 * Threepoint.kappa3 (volume : Measure ℝ)
      (anharmonicPotential lam alpha gamma)
      (fun x : ℝ => a * x + b) t
      (fun x : ℝ => a * x + b) (fun x : ℝ => a * x + b))
  Filter.atTop
    (nhds (a * a * a * (-alpha / lam ^ 3)))
```

This subsumes the base G2 theorem when $b_i = 0$ (by `add_zero`), but the two coexist in the file: the unshifted form was useful within G2’s own retrospective and is left alongside as the primer-flavoured statement.

3 Proof strategy

The structural argument is the same as G2’s: `Threepoint.kappa3` is the symmetric joint third cumulant of three observables, so

$$\kappa_3(\alpha_1, \alpha_2, \alpha_3) = a_1 a_2 a_3 \cdot \kappa_3(x, x, x)$$

follows by trilinearity in the three slots and the standard fact that κ_3 vanishes whenever any slot is constant. The Lean machinery to extract this fact from

$$\kappa_3(\text{vol}, L, A, t, \varphi, B) = \langle \varphi AB \rangle - \langle \varphi A \rangle \langle B \rangle - \langle \varphi B \rangle \langle A \rangle - \langle AB \rangle \langle \varphi \rangle + 2 \langle \varphi \rangle \langle A \rangle \langle B \rangle$$

splits into three layers.

Layer 1: bridging. At $h = 0$ the `Threepoint.gibbsExp` reduces to `Laplace.gibbsExpectation`, regardless of the perturbation direction A . Tide 9 already has this bridge as a private lemma; G2 inlined a `simp`-shaped variant of it, and so does this tide. Re-deriving locally costs five lines.

Layer 2: cubic-polynomial expansion. Each of the seven `gibbsExpectation` calls in the unfolded κ_3 has an integrand that, after `funext x; ring`, becomes a polynomial $c_3 x^3 + c_2 x^2 + c_1 x + c_0$ in x of degree at most three. A single helper reduces such an integrand to a linear combination of the four moments $\langle x^3 \rangle, \langle x^2 \rangle, \langle x \rangle, 1$, applying I2’s expectation algebra (`add`, `smul`, `const`) in sequence. The cubic helper is the bulk of the file (~ 120 lines, including the four integrability witnesses).

Layer 3: reduce to ring. After the bridging and the cubic helper fire on both sides of the desired identity, both sides are polynomials in $(a_i, b_i, m_1, m_2, m_3)$ where m_k denotes $\langle x^k \rangle$. The closing tactic is `ring`.³

The base G2 tide skipped Layer 2 entirely: with $b_i = 0$, the integrands have no constant terms, so `gibbsExpectation.smul` alone suffices and `gibbsExpectation.const` (the only lemma in I2’s algebra that carries an integrability or partition-positivity hypothesis) never fires. That is why G2’s ~ 100 lines do not need the integrability infrastructure that this tide spends ~ 80 lines establishing.

4 Roadblocks and resolutions

The race. The `tide-pick` skill claimed G2 for me at 2026-05-07T05:33:30Z as a re-rank from C1 (which a parallel session had taken at 05:25Z). I drafted candidates, consulted GPT-5.5 Pro, and started Step 3 in the worktree. Some time into Step 3, when first running `lake build`, two errors surfaced: an unrelated `rewrite` pattern miss, and a `kappa3.anharmonic.affine.asymptotic` has already been declared. The latter was the diagnostic: a parallel session had shipped G2 to the same branch (`tide/kappa3-affine-anharmonic`) before mine landed.

The parallel session’s solution was the unshifted case (bx, ax, cx) . Its retrospective explicitly identified the shifted-affine extension as a follow-up. So my work was not duplicate — it was the strict-improvement that the parallel session had deferred. The remediation: rename my theorems to `kappa3.anharmonic.shifted.affine.*` and ship the file as an additive layer on top of the parallel session’s commit. The resulting structure (G2 unshifted + this tide’s shifted extension on the same branch, neither subsuming the other in the public surface) is unusual, but correct given the order of landing.

Integrability bookkeeping. The `gibbsExpectation.const L t c hZ : <c> = c` lemma carries a `partitionFunction-non-zero` hypothesis `hZ : partitionFunction L t \ne 0`. For the anharmonic potential with $t > 0$, this follows from `integral.exp.pos`, which itself needs integrability of $\exp(-(t \cdot L_{\text{anh}}))$ over $\mathbb{R}$. That integrability is not directly in the seabed: Tide 9’s I_n / J_n machinery in `IntegralRemainder.lean` uses a substitution-based scaling argument that does not surface a `Integrable` witness for $x^n \cdot \exp(-t L_{\text{anh}})$ under the literal anharmonic measure. The fix: a `integrable.pow.mul.exp.neg.anharmonic` helper that dominates by the Gaussian $|x|^n \cdot \exp(-tcx^2)$ via `anharmonic.coercive`’s lower bound $L_{\text{anh}}(x) \geq cx^2$, then applies `Integrable.mono` against `integrable.abs.pow.mul.exp.neg.mul.sq`. About 20 lines.

The single rw that didn’t fire. The first build attempt also failed on an over-eager `rw` that tried to rewrite the bare $(\text{fun } x \Rightarrow x)$ in the `kappa3.volume.unfold-RHS` to its cubic-helper-friendly form $0 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 0$. The pattern didn’t match because the RHS already had the bare `lambda` after Layer 1’s bridging `simp`. Resolution: don’t rewrite it; `ring` handles `gibbsExpectation L t (fun x => x)` as an opaque variable, and the LHS expansions all reduce to combinations involving the same opaque term. Removing the offending `rw` fixed the build on the second attempt.

5 What was Mathlib, what was new

Mathlib. `integral.exp.pos` (positivity from strict positivity of `exp`), `Integrable.mono` (domination), `Integrable.const.mul` (scaling), `Tendsto.const.mul` and `Tendsto.congr`’ (the standard

³The symbolic identity was verified in `sympy` as part of Step 2’s numerical sanity, exactly as `ring` sees it after expansion. The Lean proof and the `sympy` script use identical algebra.

limit-massage pair from G2 and dozens of earlier laplace tides). All familiar; no new Mathlib API surfaced.

Laplace seabed (existing). Tide 9’s headline asymptotic and its $h = 0$ bridge (re-derived locally because the original is `private`), I2’s expectation algebra, the anharmonic coercivity bound, and Mathlib’s Gaussian-form integrability lemma.⁴

New. An anharmonic-side polynomial-times-exponential integrability witness, a partition-positivity corollary, a cubic-polynomial expansion helper, a local re-derivation of the κ_3 -at-volume bridge, and the two public theorems.⁵ The cubic helper has the right shape to be promoted once a second consumer materialises (e.g. a $\langle x^k \rangle_h$ tide via I4’s shift method); not yet a refactor trigger.

6 Lessons

- *Race resolution at Step 3 needs a watch on the branch.* The `tide-pick` skill’s claim system handles races at the survey level; Step 3 races (where two sessions advance the same branch independently) are not handled. The empirical resolution here was clean — the parallel session left a follow-up I picked up — but a more conflicted race (e.g. both sessions targeting the same theorem name) would have needed manual reconciliation. Worth recording but not yet automation-ripe.
- *The G2 follow-up estimate was $5\times$ low.* The retrospective for the unshifted case estimated ~ 50 lines; the actual cost was ~ 270 lines, driven entirely by the integrability bookkeeping that arises only when constant terms enter the integrand. Future tide estimates over constant-observable extensions should account for the partition-positivity and Gaussian-domination helpers (~ 80 lines of plumbing per first-time use in a fresh potential class).
- *The cubic-polynomial expansion helper is the natural “second-use” refactor target.* The pattern (rewrite integrand as $c_3x^3 + \dots + c_0$, then peel via I2’s `add / smul / const` algebra) is shape-portable to the harmonic case (C1) and to higher-degree polynomial-observable expansions (e.g. a κ_4 tide). At third use, promote to a generic potential-agnostic helper.

7 Follow-ups

- *Generic potential-agnostic shifted-affine κ_3 identity.* This tide’s private helper is fully algebraic in the moments m_1, m_2, m_3 ; the only anharmonic-specific input is the integrability witness. A generic potential-agnostic version for any L with a polynomial-up-to-degree-3 integrability bundle and positive partition function would unify this tide with C1 (the harmonic-side twin, currently in flight). About ~ 80 lines of refactoring; defer until C1 lands and the joint surface is visible.

⁴Names: Tide 9’s `kappa3.anharmonic.id.id.id.asymptotic` and `threepoint.gibbsExp.volume.zero.eq`; I2’s `gibbsExpectation.smul / _add / _const / _zero` in `Laplace.Gibbs`; `anharmonic.coercive` in `Laplace.OneD.Anharmonic`; and `integrable.abs.pow.mul.exp.neg.mul_sq` in `Laplace.OneD.IntegralRemainder`.

⁵Names: `integrable.pow.mul.exp.neg.anharmonic`, `partitionFunction.anharmonic.pos`, `gibbsExp.anharmonic.cubic.eq`, `kappa3.volume.unfold`, and the public `kappa3.anharmonic.shifted.affine.eq.smul + kappa3.anharmonic.shifted.affine.asymptotic`.

- *Promote the cubic-polynomial expansion helper.* The helper here is the natural “poly of degree ≤ 3 ” Gibbs expectation lemma. With C1’s harmonic version and possibly an N1 (generic- k J-function) extension, three consumers could materialise. Promotion target: a new `Laplace.OneD.GibbsPolyExp` module, or eventually `lean-common`.
- *Shifted-affine κ_4 .* Once a κ_4 definition lands in `Threepoint`, the same trilinear-into-quadrilinear argument should give $\kappa_4(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = a_1 a_2 a_3 a_4 \cdot \kappa_4(x, x, x, x)$. About ~ 120 lines: a quadratic expansion helper (degree ≤ 4 polynomials) plus the unfolding. Tide-shaped.

Acknowledgements

GPT-5.5 Pro’s deliberation at Step 2 contributed the “`rewrite to moments + ring`” tactic (Section 3, Layer 3), the recommendation to keep the helper local for $t > 0$ rather than all t , and the architectural advice not to over-abstract to a potential-agnostic public lemma in this tide. The full consult is preserved at `projects/primer/tide-log/gpt55_tide_kappa3_affine_anharmonic.v1.md`. GPT also recommended the exact-finite- t identity numerical check (rather than only the asymptotic), which surfaced the constants-only edge case (zero limit) and the $a_1 = 0$ case (zero everywhere) — both validated to machine precision in the `sympy + scipy` scripts.

This Tide branched off the post-G2 tip on the same Tide branch as G2 itself; the two land additively, with this tide’s commit on top of G2’s retrospective.⁶

⁶Branch: `tide/kappa3-affine-anharmonic`. G2 commits: `1b3e9bd` (proof) + `225ab8d` (retrospective).